ann-rta-final-assignment-1

April 2, 2026

1 Feed Forward Neural Network from Scratch using RTA Dataset

Objective:
Implement a simple feedforward neural network from scratch in Python without using in-built deep learning libraries.
Include forward pass, backpropagation, gradient descent, and display updated weights in each epoch.

Additional experiments: - Different hidden layer sizes - Different learning rates - Error visualization - Comparison with sklearn and keras models

1.1 Import Libraries

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

1.2 Load and Prepare RTA Dataset

[2]:
```
df = pd.read_csv("/content/RTA Dataset.csv")

# Extract hour from Time column
df['Hour'] = pd.to_datetime(df['Time'], errors='coerce').dt.hour

# Select useful columns
df = df[['Number_of_vehicles_involved',
         'Number_of_casualties',
         'Hour',
         'Accident_severity']]

df = df.dropna()

# Convert target variable
df['Accident_severity'] = df['Accident_severity'].map({
    'Slight Injury':0,
    'Serious Injury':1,
    'Fatal injury':1
})
```

```
df = df.dropna()

X = df[['Number_of_vehicles_involved','Number_of_casualties','Hour']].values
y = df['Accident_severity'].values.reshape(-1,1)

# Normalize
X = (X - X.mean(axis=0)) / X.std(axis=0)

print("Dataset shape:",X.shape)
```

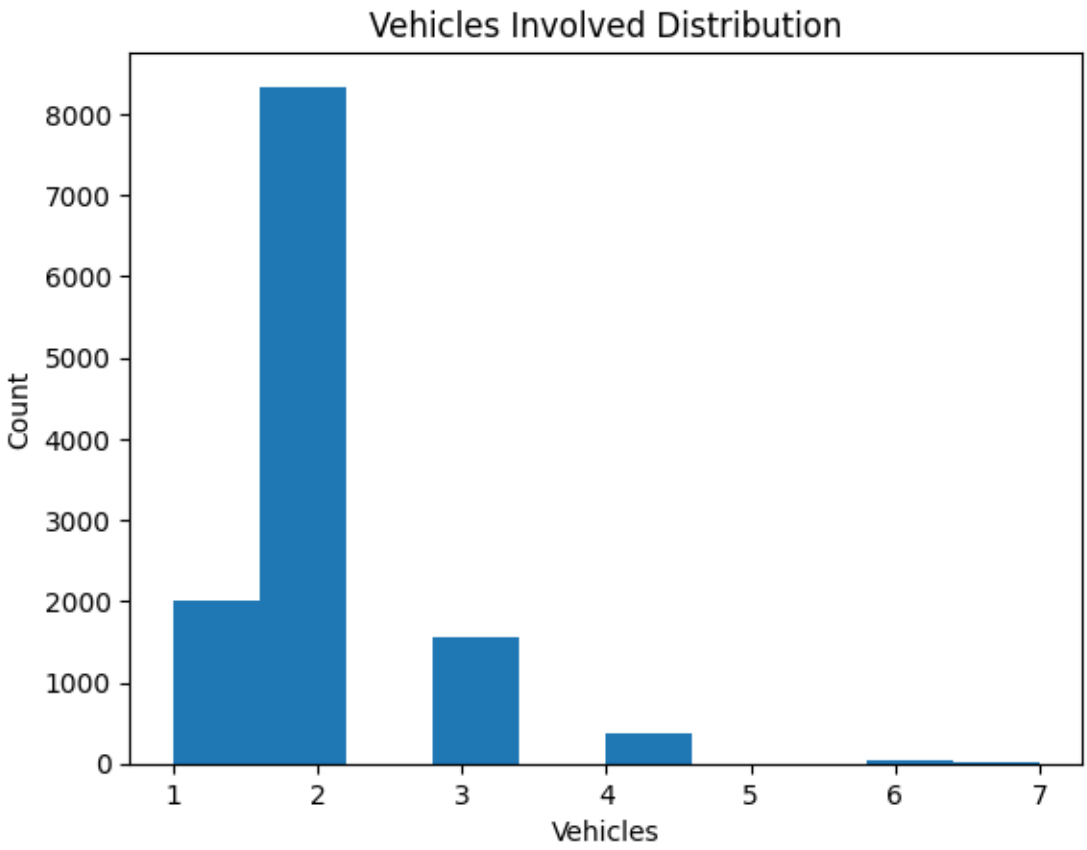
```
Dataset shape: (12316, 3)
```

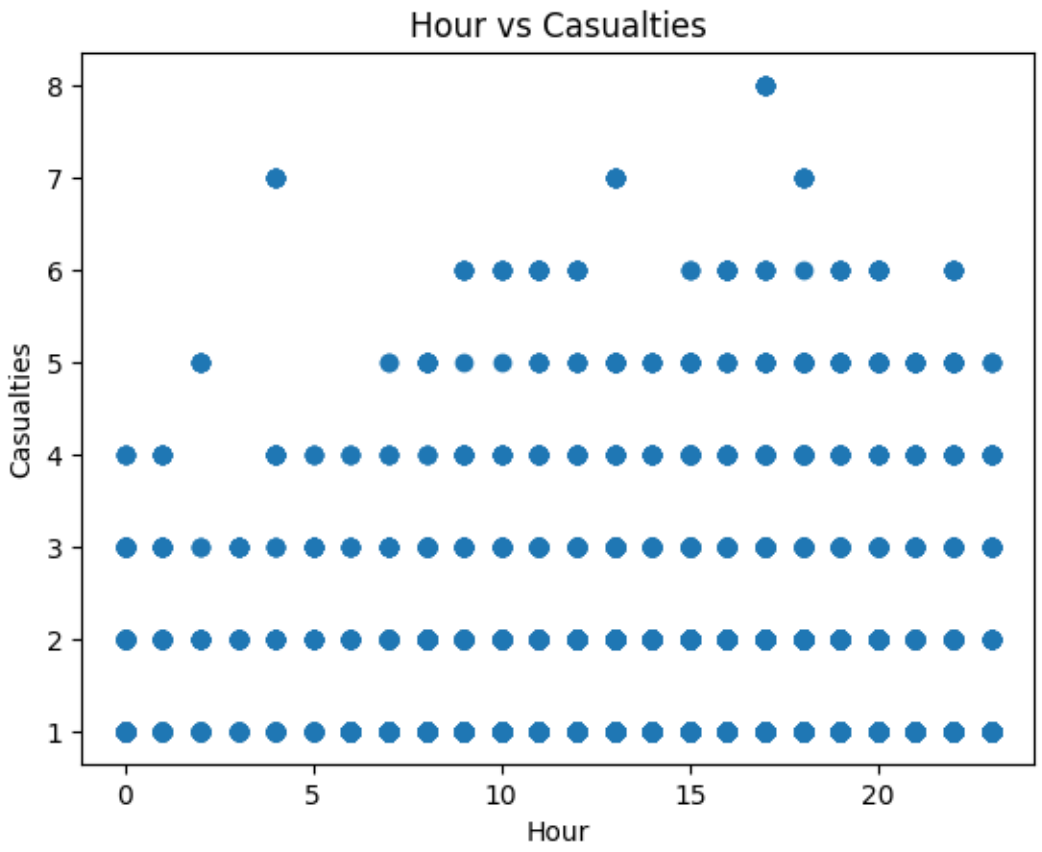
```
/tmp/ipykernel_2375/67496561.py:4: UserWarning: Could not infer format, so each
element will be parsed individually, falling back to `dateutil`. To ensure
parsing is consistent and as-expected, please specify a format.
  df['Hour'] = pd.to_datetime(df['Time'], errors='coerce').dt.hour
```

1.3 Visualization of Dataset

[3]:
```
plt.figure()
plt.hist(df['Number_of_vehicles_involved'])
plt.title("Vehicles Involved Distribution")
plt.xlabel("Vehicles")
plt.ylabel("Count")
plt.show()

plt.figure()
plt.scatter(df['Hour'],df['Number_of_casualties'])
plt.title("Hour vs Casualties")
plt.xlabel("Hour")
plt.ylabel("Casualties")
plt.show()
```


Vehicles Involved Distribution
Count
8000
7000
6000
5000
4000
3000
2000
1000
0
1
2
3
4
5
6
7
Vehicles



1.4 ANN From Scratch

[4]:
```
class SimpleANN:

    def __init__(self,input_size,hidden_size,output_size,lr):

        self.lr = lr

        self.W1 = np.random.randn(input_size,hidden_size)
        self.b1 = np.zeros((1,hidden_size))

        self.W2 = np.random.randn(hidden_size,output_size)
        self.b2 = np.zeros((1,output_size))

    def sigmoid(self,x):
        return 1/(1+np.exp(-x))

    def sigmoid_derivative(self,x):
        return x*(1-x)
```

```
    def forward(self,X):

        self.z1 = np.dot(X,self.W1) + self.b1
        self.a1 = self.sigmoid(self.z1)

        self.z2 = np.dot(self.a1,self.W2) + self.b2
        self.a2 = self.sigmoid(self.z2)

        return self.a2

    def backward(self,X,y,output):

        error = y-output
        d_output = error*self.sigmoid_derivative(output)

        error_hidden = d_output.dot(self.W2.T)
        d_hidden = error_hidden*self.sigmoid_derivative(self.a1)

        self.W2 += self.a1.T.dot(d_output)*self.lr
        self.b2 += np.sum(d_output,axis=0,keepdims=True)*self.lr

        self.W1 += X.T.dot(d_hidden)*self.lr
        self.b1 += np.sum(d_hidden,axis=0,keepdims=True)*self.lr

        return np.mean(np.square(error))

    def train(self,X,y,epochs):

        errors=[]

        for epoch in range(epochs):

            output = self.forward(X)
            loss = self.backward(X,y,output)

            errors.append(loss)

            print("\nEpoch:",epoch+1)
            print("W1 Weights")
            print(self.W1)
            print("W2 Weights")
            print(self.W2)

        return errors
```

1.5 Train Model

[5]:
```
nn = SimpleANN(input_size=3,hidden_size=5,output_size=1,lr=0.01)

errors = nn.train(X,y,epochs=10)
```

```
Epoch: 1
W1 Weights
[[-0.67026784  1.00299639  0.71187297 -1.02855891  0.57816897]
 [ 0.04083237 -1.24697343 -0.76381484 -1.19295452 -0.45087638]
 [-0.34645419  2.32132238  0.68774531 -1.40070289  1.9414281 ]]
W2 Weights
[[-5.00232688]
 [-4.56859112]
 [-5.36154825]
 [-4.19566015]
 [-3.56672524]]

Epoch: 2
W1 Weights
[[-0.67026825  1.00299636  0.71187256 -1.02855891  0.57816889]
 [ 0.04083141 -1.24697352 -0.76381598 -1.19295459 -0.4508766 ]
 [-0.34645405  2.32132238  0.68774549 -1.40070284  1.94142812]]
W2 Weights
[[-5.00232667]
 [-4.56859112]
 [-5.36154814]
 [-4.19566013]
 [-3.56672522]]

Epoch: 3
W1 Weights
[[-0.67026867  1.00299632  0.71187214 -1.02855891  0.5781688 ]
 [ 0.04083044 -1.24697361 -0.76381712 -1.19295466 -0.45087681]
 [-0.34645392  2.32132239  0.68774566 -1.40070279  1.94142813]]
W2 Weights
[[-5.00232647]
 [-4.56859111]
 [-5.36154802]
 [-4.19566011]
 [-3.5667252 ]]

Epoch: 4
W1 Weights
[[-0.67026908  1.00299629  0.71187172 -1.02855891  0.57816872]
 [ 0.04082947 -1.24697369 -0.76381827 -1.19295472 -0.45087703]
 [-0.34645379  2.32132239  0.68774583 -1.40070275  1.94142815]]
```

```
W2 Weights
[[-5.00232626]
 [-4.5685911 ]
 [-5.36154791]
 [-4.19566009]
 [-3.56672518]]

Epoch: 5
W1 Weights
[[-0.67026949  1.00299625  0.7118713  -1.02855891  0.57816863]
 [ 0.0408285  -1.24697378 -0.76381941 -1.19295479 -0.45087724]
 [-0.34645365  2.3213224   0.68774601 -1.4007027   1.94142817]]
W2 Weights
[[-5.00232606]
 [-4.56859109]
 [-5.3615478 ]
 [-4.19566007]
 [-3.56672516]]

Epoch: 6
W1 Weights
[[-0.67026991  1.00299622  0.71187088 -1.02855891  0.57816854]
 [ 0.04082754 -1.24697387 -0.76382055 -1.19295486 -0.45087746]
 [-0.34645352  2.3213224   0.68774618 -1.40070265  1.94142818]]
W2 Weights
[[-5.00232585]
 [-4.56859109]
 [-5.36154769]
 [-4.19566005]
 [-3.56672514]]

Epoch: 7
W1 Weights
[[-0.67027032  1.00299618  0.71187046 -1.02855891  0.57816846]
 [ 0.04082657 -1.24697396 -0.76382169 -1.19295492 -0.45087767]
 [-0.34645339  2.32132241  0.68774635 -1.40070261  1.9414282 ]]
W2 Weights
[[-5.00232565]
 [-4.56859108]
 [-5.36154757]
 [-4.19566003]
 [-3.56672513]]

Epoch: 8
W1 Weights
[[-0.67027074  1.00299615  0.71187004 -1.02855891  0.57816837]
 [ 0.0408256  -1.24697404 -0.76382284 -1.19295499 -0.45087789]
 [-0.34645325  2.32132241  0.68774653 -1.40070256  1.94142822]]
```

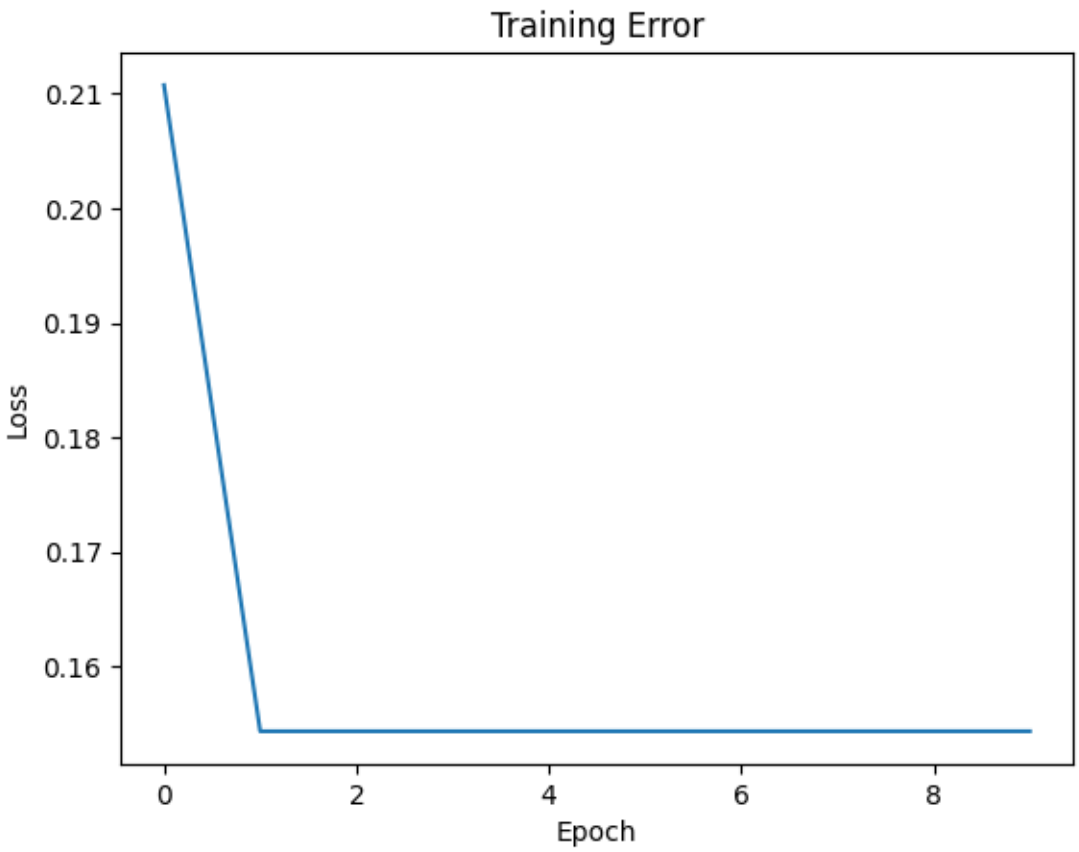
```
W2 Weights
[[-5.00232544]
 [-4.56859107]
 [-5.36154746]
 [-4.19566001]
 [-3.56672511]]

Epoch: 9
W1 Weights
[[-0.67027115  1.00299611  0.71186962 -1.02855891  0.57816829]
 [ 0.04082463 -1.24697413 -0.76382398 -1.19295506 -0.4508781 ]
 [-0.34645312  2.32132242  0.6877467  -1.40070251  1.94142824]]
W2 Weights
[[-5.00232524]
 [-4.56859106]
 [-5.36154735]
 [-4.19565999]
 [-3.56672509]]

Epoch: 10
W1 Weights
[[-0.67027156  1.00299608  0.7118692  -1.02855891  0.5781682 ]
 [ 0.04082366 -1.24697422 -0.76382512 -1.19295513 -0.45087832]
 [-0.34645298  2.32132242  0.68774687 -1.40070247  1.94142825]]
W2 Weights
[[-5.00232503]
 [-4.56859105]
 [-5.36154723]
 [-4.19565997]
 [-3.56672507]]
```

1.6 Error Curve

[6]:
```
plt.figure()
plt.plot(errors)
plt.title("Training Error")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```



1.7 Experiment with Hidden Layers

[7]:
```
for h in [3,5,8]:

    print("Hidden Layer:",h)

    model = SimpleANN(3,h,1,0.01)
    model.train(X,y,5)
```

```
Hidden Layer: 3

Epoch: 1
W1 Weights
[[-0.4976856  -1.30239141 -0.88739837]
 [-0.13438656  0.90822641 -0.19667019]
 [-1.23231152 -0.40451122 -0.03380276]]
W2 Weights
[[-5.9702484 ]
 [-5.57552411]
 [-5.89801501]]
```

```
Epoch: 2
W1 Weights
[[-0.49768786 -1.30239204 -0.8873994 ]
 [-0.13438691  0.90822575 -0.1966703 ]
 [-1.23231189 -0.40451161 -0.03380352]]
W2 Weights
[[-5.97024819]
 [-5.575524  ]
 [-5.89801482]]

Epoch: 3
W1 Weights
[[-0.49769011 -1.30239267 -0.88740043]
 [-0.13438726  0.9082251  -0.19667041]
 [-1.23231227 -0.40451201 -0.03380428]]
W2 Weights
[[-5.97024799]
 [-5.57552389]
 [-5.89801464]]

Epoch: 4
W1 Weights
[[-0.49769237 -1.3023933  -0.88740146]
 [-0.13438761  0.90822444 -0.19667053]
 [-1.23231265 -0.40451241 -0.03380504]]
W2 Weights
[[-5.97024779]
 [-5.57552377]
 [-5.89801445]]

Epoch: 5
W1 Weights
[[-0.49769463 -1.30239392 -0.88740249]
 [-0.13438796  0.90822379 -0.19667064]
 [-1.23231303 -0.40451281 -0.0338058 ]]
W2 Weights
[[-5.97024758]
 [-5.57552366]
 [-5.89801426]]
Hidden Layer: 5

Epoch: 1
W1 Weights
[[ 0.00307113  0.09501318 -0.23140043 -1.28566156 -0.45036705]
 [-0.9294539  -0.290529   -1.3881685   1.37530392 -1.93232474]
 [-0.07674034  1.49671797 -0.45639228 -0.39903183  0.41550883]]
W2 Weights
```

```
[[-2.86981504]
 [-1.93473186]
 [-2.60357667]
 [-1.80763006]
 [-1.6306687 ]]

Epoch: 2
W1 Weights
[[ 0.00225988  0.09423215 -0.23168344 -1.28597507 -0.45038841]
 [-0.9311301  -0.29200456 -1.38849457  1.37489319 -1.93234045]
 [-0.07651497  1.49678407 -0.45613521 -0.39900064  0.41552338]]
W2 Weights
[[-2.86905824]
 [-1.93377621]
 [-2.60327163]
 [-1.80598772]
 [-1.63051701]]

Epoch: 3
W1 Weights
[[ 1.44193203e-03  9.34452820e-02 -2.31968913e-01 -1.28629148e+00
  -4.50409971e-01]
 [-9.32814738e-01 -2.93490031e-01 -1.38882315e+00  1.37447837e+00
  -1.93235638e+00]
 [-7.62875863e-02  1.49684950e+00 -4.55876234e-01 -3.98969194e-01
   4.15537997e-01]]
W2 Weights
[[-2.86829729]
 [-1.93281448]
 [-2.60296481]
 [-1.80433204]
 [-1.63036454]]

Epoch: 4
W1 Weights
[[ 6.17176312e-04  9.26525092e-02 -2.32256896e-01 -1.28661082e+00
  -4.50431754e-01]
 [-9.34507910e-01 -2.94985526e-01 -1.38915427e+00  1.37405940e+00
  -1.93237254e+00]
 [-7.60581554e-02  1.49691421e+00 -4.55615329e-01 -3.98937487e-01
   4.15552677e-01]]
W2 Weights
[[-2.86753213]
 [-1.93184659]
 [-2.60265619]
 [-1.80266282]
 [-1.63021126]]
```

```
Epoch: 5
W1 Weights
[[-2.14498235e-04  9.18537619e-02 -2.32547426e-01 -1.28693314e+00
  -4.50453757e-01]
 [-9.36209708e-01 -2.96491143e-01 -1.38948797e+00  1.37363621e+00
  -1.93238893e+00]
 [-7.58266489e-02  1.49697820e+00 -4.55352467e-01 -3.98905518e-01
   4.15567421e-01]]
W2 Weights
[[-2.8667627 ]
 [-1.93087246]
 [-2.60234574]
 [-1.80097989]
 [-1.63005718]]
Hidden Layer: 8

Epoch: 1
W1 Weights
[[ 0.4754641  -0.03175908 -0.6755949  -1.31813872  2.00465764 -0.8711592
   0.82316898 -0.18437283]
 [ 0.87547752  0.5898978   0.80480303  0.47519384  0.18454442  0.07799807
   0.05377257 -1.41464195]
 [-1.16555039  0.64865793 -1.25112749 -0.41046999 -2.10701156 -0.24238346
   0.93270331 -1.20808836]]
W2 Weights
[[-5.98158607]
 [-4.88468648]
 [-5.89529479]
 [-4.93410815]
 [-3.66982285]
 [-5.4188256 ]
 [-4.84093509]
 [-6.08700952]]

Epoch: 2
W1 Weights
[[ 0.4754641  -0.03175908 -0.6755949  -1.31813872  2.00465764 -0.8711592
   0.82316898 -0.18437283]
 [ 0.87547752  0.5898978   0.80480303  0.47519384  0.18454442  0.07799807
   0.05377257 -1.41464195]
 [-1.16555039  0.64865793 -1.25112749 -0.41046999 -2.10701156 -0.24238346
   0.93270331 -1.20808836]]
W2 Weights
[[-5.98158607]
 [-4.88468648]
 [-5.89529479]
 [-4.93410815]
 [-3.66982285]
```

```
 [-5.4188256 ]
 [-4.84093509]
 [-6.08700952]]

Epoch: 3
W1 Weights
[[ 0.4754641  -0.03175908 -0.6755949  -1.31813872  2.00465764 -0.8711592
   0.82316898 -0.18437283]
 [ 0.87547752  0.5898978   0.80480303  0.47519384  0.18454442  0.07799807
   0.05377257 -1.41464195]
 [-1.16555039  0.64865793 -1.25112749 -0.41046999 -2.10701156 -0.24238346
   0.93270331 -1.20808836]]
W2 Weights
[[-5.98158607]
 [-4.88468648]
 [-5.89529479]
 [-4.93410815]
 [-3.66982285]
 [-5.4188256 ]
 [-4.84093509]
 [-6.08700952]]

Epoch: 4
W1 Weights
[[ 0.4754641  -0.03175908 -0.6755949  -1.31813872  2.00465764 -0.8711592
   0.82316898 -0.18437283]
 [ 0.87547752  0.5898978   0.80480303  0.47519384  0.18454442  0.07799807
   0.05377257 -1.41464195]
 [-1.16555039  0.64865793 -1.25112749 -0.41046999 -2.10701156 -0.24238346
   0.93270331 -1.20808836]]
W2 Weights
[[-5.98158607]
 [-4.88468648]
 [-5.89529479]
 [-4.93410815]
 [-3.66982285]
 [-5.4188256 ]
 [-4.84093509]
 [-6.08700952]]

Epoch: 5
W1 Weights
[[ 0.4754641  -0.03175908 -0.6755949  -1.31813872  2.00465764 -0.8711592
   0.82316898 -0.18437283]
 [ 0.87547752  0.5898978   0.80480303  0.47519384  0.18454442  0.07799807
   0.05377257 -1.41464195]
 [-1.16555039  0.64865793 -1.25112749 -0.41046999 -2.10701156 -0.24238346
   0.93270331 -1.20808836]]
```

```
W2 Weights
[[-5.98158607]
 [-4.88468648]
 [-5.89529479]
 [-4.93410815]
 [-3.66982285]
 [-5.4188256 ]
 [-4.84093509]
 [-6.08700952]]
```

1.8 Experiment with Learning Rates

[8]:
```
for lr in [0.1,0.01,0.001]:

    print("Learning Rate:",lr)

    model = SimpleANN(3,5,1,lr)
    model.train(X,y,5)
```

```
Learning Rate: 0.1

Epoch: 1
W1 Weights
[[ 1.8729512  -1.06171817  2.48088274  0.97843139 -0.11979568]
 [ 3.92962645  0.45298958  4.31546614  8.92475738 -2.32933915]
 [-0.33682536 -0.08305057  0.51284724 -2.9683721   2.37669221]]
W2 Weights
[[-61.66976145]
 [-62.81524528]
 [-57.69263344]
 [-57.15020553]
 [-63.90514973]]

Epoch: 2
W1 Weights
[[ 1.8729512  -1.06171817  2.48088274  0.97843139 -0.11979568]
 [ 3.92962645  0.45298958  4.31546614  8.92475738 -2.32933915]
 [-0.33682536 -0.08305057  0.51284724 -2.9683721   2.37669221]]
W2 Weights
[[-61.66976145]
 [-62.81524528]
 [-57.69263344]
 [-57.15020553]
 [-63.90514973]]

Epoch: 3
W1 Weights
[[ 1.8729512  -1.06171817  2.48088274  0.97843139 -0.11979568]
```

```
 [ 3.92962645  0.45298958  4.31546614  8.92475738 -2.32933915]
 [-0.33682536 -0.08305057  0.51284724 -2.9683721   2.37669221]]
W2 Weights
[[-61.66976145]
 [-62.81524528]
 [-57.69263344]
 [-57.15020553]
 [-63.90514973]]

Epoch: 4
W1 Weights
[[ 1.8729512  -1.06171817  2.48088274  0.97843139 -0.11979568]
 [ 3.92962645  0.45298958  4.31546614  8.92475738 -2.32933915]
 [-0.33682536 -0.08305057  0.51284724 -2.9683721   2.37669221]]
W2 Weights
[[-61.66976145]
 [-62.81524528]
 [-57.69263344]
 [-57.15020553]
 [-63.90514973]]

Epoch: 5
W1 Weights
[[ 1.8729512  -1.06171817  2.48088274  0.97843139 -0.11979568]
 [ 3.92962645  0.45298958  4.31546614  8.92475738 -2.32933915]
 [-0.33682536 -0.08305057  0.51284724 -2.9683721   2.37669221]]
W2 Weights
[[-61.66976145]
 [-62.81524528]
 [-57.69263344]
 [-57.15020553]
 [-63.90514973]]
Learning Rate: 0.01

Epoch: 1
W1 Weights
[[ 1.82266166  0.53481627 -0.43444542  0.21597948 -0.81023461]
 [ 1.67349715  0.55615688 -0.09588191  0.5765116  -0.17889769]
 [ 0.26539513 -1.53894509 -1.4297588   0.00612365 -1.92596973]]
W2 Weights
[[-6.32691268]
 [-6.29578581]
 [-5.28388985]
 [-6.53386241]
 [-7.43241123]]

Epoch: 2
W1 Weights
```

```
[[ 1.82266167  0.53481627 -0.43444542  0.21597948 -0.81023459]
 [ 1.67349715  0.55615688 -0.09588191  0.57651161 -0.17889768]
 [ 0.26539512 -1.53894509 -1.4297588   0.00612364 -1.92596975]]
W2 Weights
[[-6.32691268]
 [-6.29578581]
 [-5.28388985]
 [-6.5338624 ]
 [-7.43241123]]

Epoch: 3
W1 Weights
[[ 1.82266167  0.53481627 -0.43444542  0.21597949 -0.81023458]
 [ 1.67349715  0.55615688 -0.09588191  0.57651161 -0.17889768]
 [ 0.26539511 -1.53894509 -1.4297588   0.00612363 -1.92596977]]
W2 Weights
[[-6.32691268]
 [-6.29578581]
 [-5.28388985]
 [-6.5338624 ]
 [-7.43241123]]

Epoch: 4
W1 Weights
[[ 1.82266167  0.53481627 -0.43444542  0.2159795  -0.81023457]
 [ 1.67349716  0.55615688 -0.09588191  0.57651162 -0.17889767]
 [ 0.2653951  -1.53894509 -1.4297588   0.00612361 -1.92596979]]
W2 Weights
[[-6.32691268]
 [-6.29578581]
 [-5.28388985]
 [-6.5338624 ]
 [-7.43241122]]

Epoch: 5
W1 Weights
[[ 1.82266167  0.53481627 -0.43444542  0.21597951 -0.81023456]
 [ 1.67349716  0.55615688 -0.09588191  0.57651162 -0.17889766]
 [ 0.26539509 -1.53894509 -1.4297588   0.0061236  -1.92596981]]
W2 Weights
[[-6.32691268]
 [-6.29578581]
 [-5.28388985]
 [-6.5338624 ]
 [-7.43241122]]
Learning Rate: 0.001

Epoch: 1
```

```
W1 Weights
[[ 0.46522921 -0.50397576  0.67853038 -0.17613794 -0.37119559]
 [-0.49040801 -0.46717451  0.51305294 -0.77946714 -1.15831454]
 [-0.36563157  1.66920166 -0.13416538 -1.30349729  0.00750972]]
W2 Weights
[[ 0.0768336 ]
 [-0.45939952]
 [-0.72336491]
 [-1.32313881]
 [-0.26527527]]

Epoch: 2
W1 Weights
[[ 0.46408401 -0.49898859  0.68762857 -0.16102761 -0.36749685]
 [-0.49085022 -0.46423417  0.51650969 -0.77253444 -1.15756641]
 [-0.36621807  1.67116807 -0.1293254  -1.30005372  0.00923012]]
W2 Weights
[[ 0.06820944]
 [-0.4655129 ]
 [-0.7437601 ]
 [-1.31610572]
 [-0.26045281]]

Epoch: 3
W1 Weights
[[ 0.46311358 -0.49420461  0.6966103  -0.1465563  -0.3639936 ]
 [-0.49120541 -0.46151637  0.51976732 -0.76585939 -1.15683555]
 [-0.36669665  1.67294187 -0.12480391 -1.29706178  0.01078007]]
W2 Weights
[[ 0.06286901]
 [-0.46787256]
 [-0.76003869]
 [-1.30692877]
 [-0.25288094]]

Epoch: 4
W1 Weights
[[ 0.46224805 -0.48957359  0.70551095 -0.13255577 -0.36067413]
 [-0.4915059  -0.45897065  0.52284477 -0.75944896 -1.15614586]
 [-0.36711766  1.67459597 -0.12043424 -1.29425259  0.01221522]]
W2 Weights
[[ 0.05910001]
 [-0.4685212 ]
 [-0.77420378]
 [-1.29694892]
 [-0.24420284]]

Epoch: 5
```

```
W1 Weights
[[ 0.46145564 -0.48507055  0.71434404 -0.11894449 -0.35752908]
 [-0.49176677 -0.45656979  0.5257525  -0.7532995  -1.15550551]
 [-0.36750241  1.67616498 -0.11614078 -1.29150743  0.01356198]]
W2 Weights
[[ 0.05612884]
 [-0.46837682]
 [-0.78716812]
 [-1.2867667 ]
 [-0.23515341]]
```

1.9 Sklearn Neural Network

[9]:
```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

model = MLPClassifier(hidden_layer_sizes=(5,),max_iter=500)
model.fit(X_train,y_train.ravel())

pred = model.predict(X_test)

print("Sklearn Accuracy:",accuracy_score(y_test,pred))
```

```
Sklearn Accuracy: 0.8486201298701299
```

1.10 Keras Neural Network

[10]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(5,input_dim=3,activation='relu'))
model.add(Dense(1,activation='sigmoid'))

model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])

model.fit(X,y,epochs=10,batch_size=10)
```

```
Epoch 1/10
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
1232/1232            3s 2ms/step -
accuracy: 0.8046 - loss: 0.5121
Epoch 2/10
1232/1232            2s 2ms/step -
accuracy: 0.8456 - loss: 0.4278
Epoch 3/10
1232/1232            2s 2ms/step -
accuracy: 0.8456 - loss: 0.4218
Epoch 4/10
1232/1232            2s 2ms/step -
accuracy: 0.8456 - loss: 0.4196
Epoch 5/10
1232/1232            2s 2ms/step -
accuracy: 0.8456 - loss: 0.4188
Epoch 6/10
1232/1232            2s 2ms/step -
accuracy: 0.8456 - loss: 0.4184
Epoch 7/10
1232/1232            2s 2ms/step -
accuracy: 0.8456 - loss: 0.4180
Epoch 8/10
1232/1232            2s 2ms/step -
accuracy: 0.8456 - loss: 0.4179
Epoch 9/10
1232/1232            2s 2ms/step -
accuracy: 0.8456 - loss: 0.4175
Epoch 10/10
1232/1232            3s 2ms/step -
accuracy: 0.8456 - loss: 0.4175
```

[10]: `<keras.src.callbacks.history.History at 0x7e8d49b64620>`

1.11 Conclusion

- ANN implemented from scratch
- Forward pass and backpropagation implemented manually
- Weight updates printed each epoch
- Experiments performed with different parameters
- Compared with sklearn and keras implementations